

LAB PROGRAM – TOPIC 5

SHAHANAJ C

192372292

**SIMATS Online Programming Lab**

CHERUVU SHAHANAJ
192372292RA

LAB PROGRAM - TOPIC 6

Python C C++ Java

QUESTIONS

Q1 Merge Sort with Comparison Count
10 marks

Q2 Quick Sort with Comparison Count
10 marks

Q3 Merge Sort vs Quick Sort Comparison
10 marks

Q4 Merge Sort Execution Time Measurement
10 marks

Q5 Best and Worst Case Analysis of Quick Sort
10 marks

Q6 Hybrid Merge Sort with Insertion Sort
10 marks

Q7 Counting Inversions using Merge Sort
10 marks

Q8 Quick Sort Recursion Depth Analysis
10 marks

Q9 External Merge Process Simulation
10 marks

Q1 Merge Sort with Comparison Count 10 marks

Implement the Merge Sort algorithm to sort a given array of integers. The program should recursively divide the array into smaller subarrays and then merge them in a sorted manner. While merging, count the number of element comparisons performed. Ensure that the comparison counter is incremented at every comparison step. Finally, print the sorted array along with the total number of comparisons made during execution.

Test Cases:
Input: N = 8, a[] = {12,4,78,23,45,67,89,1}
Output: 1,4,12,23,45,67,78,89
Comparisons: 17
Input: N = 6, a[] = {5,2,9,1,6,3}
Output: 1,2,3,5,6,9
Comparisons: 11

Your Code

```
1 def merge_sort(arr):  
2     comparison_count = 0  
3  
4     def sort(sub_arr):  
5         nonlocal comparison_count  
6         if len(sub_arr) <= 1:  
7             return sub_arr  
8  
9         mid = len(sub_arr) // 2  
10        left = sort(sub_arr[:mid])  
11        right = sort(sub_arr[mid:])  
12  
13        return merge(left, right)  
14  
15    def merge(left, right):  
16        nonlocal comparison_count  
17        merged = []  
18        i = j = 0  
19  
20        while i < len(left) and j < len(right):  
21            comparison_count += 1  
22            if left[i] <= right[j]:  
23                merged.append(left[i])  
24
```

Input

```
6  
5 2 9 1 6 3
```

Output

```
1,2,3,5,6,9  
Comparisons: 10
```

**SIMATS Online Programming Lab**

CHERUVU SHAHANAJ
192372292RA

LAB PROGRAM - TOPIC 6

Python C C++ Java

QUESTIONS

Q1 Merge Sort with Comparison Count
10 marks

Q2 Quick Sort with Comparison Count
10 marks

Q3 Merge Sort vs Quick Sort Comparison
10 marks

Q4 Merge Sort Execution Time Measurement
10 marks

Q5 Best and Worst Case Analysis of Quick Sort
10 marks

Q6 Hybrid Merge Sort with Insertion Sort
10 marks

Q7 Counting Inversions using Merge Sort
10 marks

Q8 Quick Sort Recursion Depth Analysis
10 marks

Q9 External Merge Process Simulation
10 marks

Q2 Quick Sort with Comparison Count 10 marks

Write a program to implement the Quick Sort algorithm using the last element as the pivot. The program should partition the array based on the pivot and recursively sort the subarrays. Count the number of comparisons made during the partitioning process and recursive calls. Ensure that the count is accurate for all cases. Display the sorted array along with the total number of comparisons.

Test Cases:
Input: N = 7, a[] = {38,27,43,3,9,82,10}
Output: 3,9,10,27,38,43,82
Comparisons: 15
Input: N = 5, a[] = {10,7,8,9,1}
Output: 1,7,8,9,10
Comparisons: 10

Your Code

```
1 def partition(arr, low, high):  
2     pivot = arr[high]  
3     i = low - 1  
4     comparisons = 0  
5     for j in range(low, high):  
6         comparisons += 1  
7         if arr[j] <= pivot:  
8             i += 1  
9             arr[i], arr[j] = arr[j], arr[i]  
10    arr[i + 1], arr[high] = arr[high], arr[i + 1]  
11    return i + 1, comparisons  
12 def quicksort(arr, low, high):  
13     comparisons = 0  
14     if low < high:  
15         p, comp = partition(arr, low, high)  
16         comparisons += comp  
17         comparisons += quicksort(arr, low, p - 1)  
18         comparisons += quicksort(arr, p + 1, high)  
19  
20     return comparisons  
21 def quick_sort_with_comparisons(arr):  
22     a = arr[:] 23     total_comparisons = quicksort(a, 0, len(a) - 1)  
24     return a, total_comparisons  
25 if __name__ == "__main__":
```

Input

```
7  
12 34 56 78 98 4 6
```

Output

```
3,9,10,27,38,43,82  
Comparisons: 11  
1,7,8,9,10  
Comparisons: 10
```

LAB PROGRAM -TOPIC 6

[← Back](#)Language: [Python](#) [C](#) [C++](#) [Java](#)

QUESTIONS

Q1
Merge Sort with Comparison
Count
10 marks**Q2**
Quick Sort with Comparison
Count
10 marks**Q3**
Merge Sort vs Quick Sort
Comparison
10 marks**Q4**
Merge Sort Execution Time
Measurement
10 marks**Q5**
Best and Worst Case Analysis of
Quick Sort
10 marks**Q6**
Hybrid Merge Sort with
Insertion Sort
10 marks**Q7**
Counting Inversions using
Merge Sort
10 marks**Q8**
Quick Sort Recursion Depth
Analysis
10 marks**Q9**
External Merge Process
Simulation
10 marks**Q3 Merge Sort vs Quick Sort Comparison**

10 marks

Develop a program that implements both Merge Sort and Quick Sort in a single program. The same input array should be used for both algorithms to ensure fairness in comparison. Count the number of comparisons separately for each algorithm. After sorting, display the sorted array along with the comparison counts of both methods. Analyze which algorithm performs better for the given input.

Test Cases:

Input N = 6, a[] = {12,11,13,5,6,7}

Output: Sorted Array : 5,6,7,11,12,13; Merge Comparisons : 11; Quick Comparisons : 12

Input N = 5, a[] = {4,2,6,1,3}

Output: Sorted Array : 1,2,3,4,6; Merge Comparisons : 8; Quick Comparisons : 9

Your Code

```
1 class SortingComparison:
2     def __init__(self):
3         self.merge_comparisons = 0
4         self.quick_comparisons = 0
5
6     def merge_sort(self, arr):
7         if len(arr) > 1:
8             mid = len(arr) // 2
9             L = arr[:mid]
10            R = arr[mid:]
11
12            self.merge_sort(L)
13            self.merge_sort(R)
14
15            i = j = k = 0
16
17            while i < len(L) and j < len(R):
18                self.merge_comparisons += 1
19                if L[i] < R[j]:
20                    arr[k] = L[i]
21                    i += 1
22                else:
23                    arr[k] = R[j]
24                    j += 1
25                k += 1
26
27            while i < len(L):
28                arr[k] = L[i]
29                i += 1
30                k += 1
31
32            while j < len(R):
33                arr[k] = R[j]
34                j += 1
35                k += 1
```

Input

```
5
12 4 76 3 4
```

Output

```
Sorted Array : 2, 4, 4, 12, 76;
Merge Comparisons : 8; Quick
Comparisons : 6
```

LAB PROGRAM -TOPIC 6

[← Back](#)Language: [Python](#) [C](#) [C++](#) [Java](#)

QUESTIONS

Q1
Merge Sort with Comparison
Count
10 marks**Q2**
Quick Sort with Comparison
Count
10 marks**Q3**
Merge Sort vs Quick Sort
Comparison
10 marks**Q4**
Merge Sort Execution Time
Measurement
10 marks**Q5**
Best and Worst Case Analysis of
Quick Sort
10 marks**Q6**
Hybrid Merge Sort with
Insertion Sort
10 marks**Q7**
Counting Inversions using
Merge Sort
10 marks**Q8**
Quick Sort Recursion Depth
Analysis
10 marks**Q9**
External Merge Process
Simulation
10 marks**Q4 Merge Sort Execution Time Measurement**

10 marks

Write a program to measure the execution time of the Merge Sort algorithm. Use appropriate timing functions to record the start and end time of execution. Apply the algorithm on different types of inputs such as sorted and reverse sorted arrays. Display the sorted output along with the time taken for execution. Analyze how input order affects performance.

Test Cases:

Input N = 7, a[] = {9,8,7,6,5,4,3}

Output: 3,4,5,6,7,8,9

Time Taken : ~0.00002 sec

Input N = 6, a[] = {1,2,3,4,5,6}

Output: 1,2,3,4,5,6

Time Taken : ~0.00002 sec

Your Code

```
1 import time
2
3 def merge_sort(arr):
4     if len(arr) > 1:
5         mid = len(arr) // 2
6         L = arr[:mid]
7         R = arr[mid:]
8
9         merge_sort(L)
10        merge_sort(R)
11
12        i = j = k = 0
13
14        while i < len(L) and j < len(R):
15            if L[i] < R[j]:
16                arr[k] = L[i]
17                i += 1
18            else:
19                arr[k] = R[j]
20                j += 1
21            k += 1
22
23        while i < len(L):
24            arr[k] = L[i]
25            i += 1
26            k += 1
27
28        while j < len(R):
29            arr[k] = R[j]
30            j += 1
31            k += 1
```

Input

```
7
9 8 7 6 5 4 3
```

Output

```
--- Testing Reverse Sorted
Array ---
Input: [9, 8, 7, 6, 5, 4, 3]
Output: [3, 4, 5, 6, 7, 8, 9]
Time Taken : ~0.000024 sec
```

LAB PROGRAM -TOPIC 6

← Back

Language: Python C C++ Java

QUESTIONS

Q1 Merge Sort with Comparison Count 10 marks

Q2 Quick Sort with Comparison Count 10 marks

Q3 Merge Sort vs Quick Sort Comparison 10 marks

Q4 Merge Sort Execution Time Measurement 10 marks

Q5 Best and Worst Case Analysis of Quick Sort 10 marks

Q6 Hybrid Merge Sort with Insertion Sort 10 marks

Q7 Counting Inversions using Merge Sort 10 marks

Q8 Quick Sort Recursion Depth Analysis 10 marks

Q9 External Merge Process Simulation 10 marks

Q5 Best and Worst Case Analysis of Quick Sort

10 marks

Create a program to analyze the best and worst-case scenarios of Quick Sort. Use a sorted array as the best case and a reverse sorted array as the worst case. Count the number of comparisons for both cases. Display the sorted output along with comparison counts. Compare the efficiency between the two cases.

Test Cases:

Best case - Input: N = 5, a[] = [1,2,3,4,5]

Output: 1,2,3,4,5

Comparisons : 6

Worst Case - Input: N = 5, a[] = [5,4,3,2,1]

Output: 1,2,3,4,5

Comparisons : 10

Your Code

```
1 def partition(arr, low, high, counter):
2     pivot = arr[high]
3     i = low - 1
4     for j in range(low, high):
5         counter[0] += 1
6         if arr[j] <= pivot:
7             i += 1
8             arr[i], arr[j] = arr[j], arr[i]
9     arr[i + 1], arr[high] = arr[high], arr[i + 1]
10    return i + 1
11
12 def quicksort(arr, low, high, counter):
13     if low < high:
14         pi = partition(arr, low, high, counter)
15         quicksort(arr, low, pi - 1, counter)
16         quicksort(arr, pi + 1, high, counter)
17
18 def run_test(arr):
19     best_worst_counter = [0]
20     a = arr[:]
21     quicksort(a, 0, len(a) - 1, best_worst_counter)
22     return a, best_worst_counter[0]
23
24
25 a = 5
```

Input

5
1 2 3 4 5

Output

```
Best Case Output: [1, 2, 3, 4, 5]
Comparisons : 10
Worst Case Output: [1, 2, 3, 4, 5]
Comparisons : 10
```

LAB PROGRAM -TOPIC 6

← Back

Language: Python C C++ Java

QUESTIONS

Q1 Merge Sort with Comparison Count 10 marks

Q2 Quick Sort with Comparison Count 10 marks

Q3 Merge Sort vs Quick Sort Comparison 10 marks

Q4 Merge Sort Execution Time Measurement 10 marks

Q5 Best and Worst Case Analysis of Quick Sort 10 marks

Q6 Hybrid Merge Sort with Insertion Sort 10 marks

Q7 Counting Inversions using Merge Sort 10 marks

Q8 Quick Sort Recursion Depth Analysis 10 marks

Q9 External Merge Process Simulation 10 marks

Q6 Hybrid Merge Sort with Insertion Sort

10 marks

Modify the Merge Sort algorithm such that it switches to Insertion Sort when the subarray size becomes small. Define a threshold value to determine when to switch. This hybrid approach should improve performance for small inputs. Print the sorted array after execution. Compare results with standard Merge Sort.

Test Cases:

Input: N = 8, a[] = [12,11,13,5,6,7,3,2]

Output: 2,3,5,6,7,11,12,13

Input: N = 6, a[] = [9,4,6,2,8,1]

Output: 1,2,4,6,8,9

Your Code

```
1 def insertion_sort(arr, left, right):
2     for i in range(left + 1, right + 1):
3         key = arr[i]
4         j = i - 1
5         while j >= left and arr[j] > key:
6             arr[j + 1] = arr[j]
7             j -= 1
8         arr[j + 1] = key
9
10 def merge(arr, left, mid, right):
11     n1 = mid - left + 1
12     n2 = right - mid
13
14     L = arr[left:mid + 1]
15     R = arr[mid + 1:right + 1]
16
17     i = 0
18     j = 0
19     k = left
20
21     while i < n1 and j < n2:
22         if L[i] <= R[j]:
23             arr[k] = L[i]
24             i += 1
25         else:
26             arr[k] = R[j]
27             j += 1
28         k += 1
```

Input

6
9 4 6 2 8 1

Output

1,2,4,6,8,9



SIMATS Online Programming Lab

DAA PROGRAMMING

CHERUVU SHAHANAJ
192372292RA

LAB PROGRAM -TOPIC 6

← Back

Language: Python C C++ Java

QUESTIONS

Q1Merge Sort with Comparison Count10 marks

Q2Quick Sort with Comparison Count10 marks

Q3Merge Sort vs Quick Sort Comparison10 marks

Q4Merge Sort Execution Time Measurement10 marks

Q5Best and Worst Case Analysis of Quick Sort10 marks

Q6Hybrid Merge Sort with Insertion Sort10 marks

Q7Counting Inversions using Merge Sort10 marks

Q8Quick Sort Recursion Depth Analysis10 marks

Q9External Merge Process Simulation10 marks

Q7 Counting Inversions using Merge Sort10 marks

Develop a program to count the number of inversions in an array using Merge Sort. An inversion is defined as a pair (i, j) such that $i < j$ and $a[i] > a[j]$. Modify the merge step to count such inversions efficiently. Print both the sorted array and the inversion count. This helps measure how far the array is from being sorted.

Test Cases:
Input: $N = 5, a[] = [2, 4, 1, 3, 5]$
Output: Sorted : 1,2,3,4,5; Inversions : 3
Input: $N = 4, a[] = [4, 3, 2, 1]$
Output: Sorted : 1,2,3,4; Inversions : 6

Your Code

```
1 def merge_and_count(arr, temp, left, mid, right):
2     i = left
3     j = mid + 1
4     k = left
5     inv_count = 0
6
7     while i <= mid and j <= right:
8         if arr[i] <= arr[j]:
9             temp[k] = arr[i]
10            i += 1
11        else:
12            temp[k] = arr[j]
13            inv_count += (mid - i + 1)
14            j += 1
15        k += 1
16    while i <= mid:
17        temp[k] = arr[i]
18        i += 1
19        k += 1
20    while j <= right:
21        temp[k] = arr[j]
22        j += 1
23        k += 1
24    for idx in range(left, right + 1):
25        arr[idx] = temp[idx]
26    return inv_count
27 def merge_sort_and_count(arr, temp, left, right):
28     if left < right:
```

Input

5
2 4 1 3 5

Output

Sorted : 1,2,3,4,5
Inversions : 3



SIMATS Online Programming Lab

DAA PROGRAMMING

CHERUVU SHAHANAJ
192372292RA

LAB PROGRAM -TOPIC 6

← Back

Language: Python C C++ Java

QUESTIONS

Q1Merge Sort with Comparison Count10 marks

Q2Quick Sort with Comparison Count10 marks

Q3Merge Sort vs Quick Sort Comparison10 marks

Q4Merge Sort Execution Time Measurement10 marks

Q5Best and Worst Case Analysis of Quick Sort10 marks

Q6Hybrid Merge Sort with Insertion Sort10 marks

Q7Counting Inversions using Merge Sort10 marks

Q8Quick Sort Recursion Depth Analysis10 marks

Q9External Merge Process Simulation10 marks

Q8 Quick Sort Recursion Depth Analysis10 marks

Implement Quick Sort and track the recursion depth during execution. Maintain a variable to store the maximum depth reached. Print the sorted array along with the maximum recursion depth. This helps analyze space complexity. Compare depth for different inputs.

Test Cases:
Input: $N = 6, a[] = [10, 7, 8, 9, 1, 5]$
Output: 1,5,7,8,9,10
Max Depth : 4
Input: $N = 5, a[] = [1, 2, 3, 4, 5]$
Output: 1,2,3,4,5
Max Depth : 5

Your Code

```
1 def partition(arr, low, high):
2     pivot = arr[high]
3     i = low - 1
4     for j in range(low, high):
5         if arr[j] <= pivot:
6             i += 1
7             arr[i], arr[j] = arr[j], arr[i]
8     arr[i + 1], arr[high] = arr[high], arr[i + 1]
9     return i + 1
10
11 def quicksort_with_depth(arr):
12     max_depth = 0
13
14     def qs(low, high, depth):
15         nonlocal max_depth
16         if low >= high:
17             return
18         max_depth = max(max_depth, depth)
19
20         p = partition(arr, low, high)
21         qs(low, p - 1, depth + 1)
22         qs(p + 1, high, depth + 1)
23
24     qs(0, len(arr) - 1, 1)
25
26     return arr, max_depth
```

Input

6
10 7 8 9 1 5

Output

1,5,7,8,9,10
Max Depth : 4



SIMATS Online Programming Lab
DAA PROGRAMMING



LAB PROGRAM -TOPIC 6

Language: Python C C++ Java

QUESTIONS

Q1Merge Sort with Comparison Count10 marks

Q2Quick Sort with Comparison Count10 marks

Q3Merge Sort vs Quick Sort Comparison10 marks

Q4Merge Sort Execution Time Measurement10 marks

Q5Best and Worst Case Analysis of Quick Sort10 marks

Q6Hybrid Merge Sort with Insertion Sort10 marks

Q7Counting Inversions using Merge Sort10 marks

Q8Quick Sort Recursion Depth Analysis10 marks

Q9External Merge Process Simulation10 marks

Q9 External Merge Process Simulation10 marks

Write a program to simulate the merge step of external sorting. Given two already sorted arrays, merge them into a single sorted array. Do not use built-in sorting functions. Print the final merged output. This represents the core step of Merge Sort.

Test Cases:
Input: a[] = {1,3,5}, b[] = {2,4,6}
Output: 1,2,3,4,5,6
Input: a[] = {10,20}, b[] = {5,15,25}
Output: 5,10,15,20,25

Your Code

```
1 def external_merge(a, b):
2     i = j = 0
3     merged = []
4
5     while i < len(a) and j < len(b):
6         if a[i] <= b[j]:
7             merged.append(a[i])
8             i += 1
9         else:
10            merged.append(b[j])
11            j += 1
12
13    while i < len(a):
14        merged.append(a[i])
15        i += 1
16
17    while j < len(b):
18        merged.append(b[j])
19        j += 1
20
21    return merged
22
23
24 a1 = [1, 3, 5]
25 b1 = [2, 4, 6]
26 print("external_merge(a1, b1), sep=",)
27
28 a2 = [10, 20]
```

Input

1 3 5
2 4 6

Output

1,2,3,4,5,6
5,10,15,20,25



SIMATS Online Programming Lab
DAA PROGRAMMING



LAB PROGRAM -TOPIC 6

Language: Python C C++ Java

QUESTIONS

Q1Merge Sort with Comparison Count10 marks

Q2Quick Sort with Comparison Count10 marks

Q3Merge Sort vs Quick Sort Comparison10 marks

Q4Merge Sort Execution Time Measurement10 marks

Q5Best and Worst Case Analysis of Quick Sort10 marks

Q6Hybrid Merge Sort with Insertion Sort10 marks

Q7Counting Inversions using Merge Sort10 marks

Q8Quick Sort Recursion Depth Analysis10 marks

Q9External Merge Process Simulation10 marks

Q10 Space Complexity Comparison (Merge vs Quick Sort)10 marks

Develop a program to compare auxiliary space usage of Merge Sort and Quick Sort. Track extra memory used during execution. Print sorted output along with space usage. Analyze which algorithm is more space-efficient. Use same input for both methods.

Test Cases:
Input: N = 5, a[] = {5,3,8,4,2}
Output: Sorted : 2,3,4,5,8; Merge Space : ~160 bytes; Quick Space : ~80 bytes
Input: N = 6, a[] = {9,7,5,3,1,2}
Output: Sorted : 1,2,3,5,7,9; Merge Space : ~192 bytes; Quick Space : ~96 bytes

Your Code

```
1 def merge_sort_with_space(arr):
2
3     space = 0
4
5     def merge_sort(a):
6         nonlocal space
7         n = len(a)
8         if n <= 1:
9             return a
10
11        mid = n // 2
12        left = merge_sort(a[:mid])
13        right = merge_sort(a[mid:])
14        space += n
15
16        i = j = 0
17        merged = []
18        while i < len(left) and j < len(right):
19            if left[i] <= right[j]:
20                merged.append(left[i])
21                i += 1
22            else:
23                merged.append(right[j])
24                j += 1
25
26        while i < len(left):
27            merged.append(left[i])
28            i += 1
29        while j < len(right):
```

Input

5
5 3 8 4 2

Output

Test Case 1:
Sorted : 2, 3, 4, 5, 8; Merge
Space : ~192 bytes; Quick Space
: ~48 bytes
Conclusion: Quick Sort is more
space-efficient for this input